

Projetos em Python para Prática e Apresentação

Lista organizada de projetos para praticar Python, evoluindo dos fundamentos até aplicações mais completas para uma disciplina de Ciência da Computação.

1. Projetos — Iniciante/Fáceis

Ideais para treinar sintaxe, condicionais, loops, funções, listas, dicionários, strings e entrada/saída.

Calculadora — Operações básicas, menu, condicionais, funções e while.

Conversor de unidades — Temperatura, distância, peso e outras conversões.

Jogo de adivinhação — Número aleatório, tentativas e dicas de maior/menor.

Pedra, Papel e Tesoura — Jogador contra computador, pontuação e random.

Gerador de senha — Gerar senhas aleatórias com tamanho definido pelo usuário.

Lista de tarefas (To-Do List) — Adicionar, remover, concluir e listar tarefas.

Agenda de contatos — Cadastrar, editar, remover e pesquisar contatos.

Sistema de notas — Cadastrar alunos, notas, médias e situação.

Caixa eletrônico — Saldo, depósito, saque e extrato simples.

Cronômetro/temporizador — Contagem regressiva usando a biblioteca time.

Contador de palavras — Contar palavras, caracteres e linhas de um texto.

Quiz — Perguntas de múltipla escolha e sistema de pontuação.

2. Projetos — Médios

Projetos para começar a trabalhar com arquivos, banco de dados, APIs, orientação a objetos e bibliotecas externas.

Sistema de biblioteca — Livros, usuários, empréstimos e devoluções.

Sistema financeiro pessoal — Receitas, despesas, categorias, saldo e SQLite.

Sistema de estoque — Produtos, quantidade, entrada, saída e estoque baixo.

Sistema de cadastro com SQLite — CRUD completo com banco de dados.

API REST com FastAPI — Endpoints GET, POST, PUT e DELETE com persistência.

Sistema de login — Cadastro, autenticação, senhas e níveis de usuário.

Web scraper — Coletar informações de sites e salvar em CSV/JSON.

Consulta de CEP — Consumir uma API e retornar informações de endereço.

Aplicação de previsão do tempo — Consultar API meteorológica e exibir dados.

Analizador de arquivos CSV — Usar Pandas para estatísticas e gráficos.

Sistema de chamados — Chamados, prioridades, categorias, status e histórico.

Gerenciador de filmes — Filmes, notas, gêneros, pesquisa e estatísticas.

Controle de gastos com gráficos — Categorias, gastos mensais e dashboard.

Chat simples — Comunicação cliente/servidor usando sockets.

Sistema de análise de dados — Limpeza, estatísticas e visualização de datasets.

3. Projetos para apresentar em uma disciplina

Aqui o foco é criar uma solução com um problema claro, tecnologia, funcionamento e resultados. A dificuldade não é o principal critério.

Sistema de detecção de fraudes — Analisar transações e calcular um score de risco usando estatística e/ou machine learning. Tecnologias possíveis: Python, FastAPI, SQL, Pandas e Scikit-learn.

Sistema de recomendação de filmes — Recomendar filmes com base em gêneros, avaliações ou similaridade. Pode usar Pandas, Scikit-learn, SQLite e FastAPI.

Dashboard de análise de dados — Transformar um dataset em informações visuais sobre vendas, produtos, regiões e faturamento. Pode usar Pandas, Matplotlib/Plotly e Streamlit.

Sistema de gerenciamento acadêmico — Gerenciar alunos, professores, disciplinas, notas, frequência, cursos e turmas.

Sistema de chamados de TI — Simular um ambiente empresarial com usuários, técnicos, prioridades, categorias, histórico e dashboard.

Sistema de análise de currículo — Identificar tecnologias, competências e palavras-chave e comparar um currículo com uma vaga.

Estoque inteligente — Controlar estoque e tentar prever quais produtos precisam de reposição.

Detector de anomalias — Encontrar comportamentos fora do padrão em transações, sensores, vendas ou outros dados.

Monitoramento de computadores — Coletar CPU, RAM, armazenamento, processos e rede e apresentar os dados em dashboard.

Sistema de análise de notícias — Coletar, processar e classificar notícias por categorias.

API de gerenciamento de biblioteca — Criar uma API profissional com FastAPI, banco, autenticação, CRUD e Swagger.

Previsão de desempenho acadêmico — Analisar notas, frequência e atividades para estimar risco de baixo desempenho.

Análise de sentimentos — Classificar textos como positivos, negativos ou neutros usando processamento de linguagem.

Segurança para login — Monitorar tentativas de login e identificar comportamentos suspeitos.

Plataforma de análise empresarial — Integrar banco, processamento Python e dashboard para analisar vendas, clientes, produtos e faturamento.

Caminho recomendado de evolução

Fundamentos: Calculadora → Adivinhação → Pedra, Papel e Tesoura → Quiz → To-Do List.

Lógica e estruturas: Agenda → Caixa eletrônico → Sistema de notas → Estoque → Sistema financeiro.

Python aplicado: CSV + Pandas → API de CEP → API de clima → Web Scraper → SQLite.

Backend: FastAPI → CRUD + banco → Sistema de chamados → Biblioteca → Login.

Projeto final: Detecção de anomalias/fraudes, análise de dados ou chamados de TI.

Sugestões mais fortes para apresentação

Para uma apresentação de Ciência da Computação, as opções que mais permitem demonstrar programação, banco de dados, APIs, análise de dados e aplicação prática são:

Detecção de anomalias/fraudes

Sistema de recomendação

Dashboard de análise de dados

Sistema de chamados de TI